

NeuroLink ML Presentation Guide

This guide is designed to help you present the machine learning components of the **NeuroLink / NeuroVerse** student wellness platform to your course teacher. It outlines the datasets used, the model training pipeline, the model selection process, and how they integrate into the live application.

1. Dataset Catalog

Your ML pipeline utilizes 5 specific CSV files located in `ml-service/Datasets/Datasets CSV/`. Here is a detailed breakdown of each:

Dataset File	Size (Rows × Columns)	Core Target / Purpose	Key Columns & Features	Preprocessing Applied
Student Mental health.csv	101 × 11	Mood Prediction Model	Age, Choose your gender, Your current year of Study, What is your CGPA?, Do you have Depression?, Do you have Anxiety?, Do you have Panic attack?	Numerical mapping of categorical features, derivation of continuous <code>mood_score</code> , and StandardScaler normalization.
Combined Data.csv	53,043 × 3	Sentiment Classifier	<code>statement</code> (renamed to <code>text</code>), <code>status</code> (mapped to sentiment labels: NEUTRAL, NEGATIVE, CRISIS)	Lowercasing, removal of punctuation/numbers, TF-IDF vectorization.
mental_health.csv	27,977 × 2	Sentiment Classifier (Crisis)	<code>text</code> , <code>label</code> (mapped to sentiment labels: 0 $\rightarrow$ NEUTRAL, 1 $\rightarrow$ CRISIS)	Combined with the above dataset, cleaned, and vectorized.

Sleep_health_and_lifestyle_dataset.csv	Sleep Insights & Analytics	Sleep Duration, Quality of Sleep, Stress Level, Occupation, Sleep Disorder	Handled missing values (filled nulls in Sleep Disorder with 'None'), calculated correlations and generated rule-based thresholds.
Mental_Health_FAQ.csv	FAQ Knowledge Base	Question_ID, Questions, Answers	Lowercasing, punctuation stripping, stopword removal to extract high-relevance search keywords.

■ ■ 2. The Training Pipeline (train_all_models.py)

When you run `python training/train_all_models.py`, it executes four distinct tasks sequentially. For each task, the pipeline compares multiple algorithms and saves the best-performing one to `ml-service/models/`.

```
Datasets --> train_all_models.py
    ■ ■ ■ Task 1: Mood Prediction (Linear Regression, Random Forest, Gradient Boosting)
    ■ ■ ■ Task 2: Sentiment Classifier (Logistic Regression, Random Forest, SVM)
    ■ ■ ■ Task 3: Sleep Insights (Rule-based Thresholds & Pearson Correlations)
    ■ ■ ■ Task 4: FAQ Knowledge Base (Keyword Search Index)
```

Task 1: Mood Prediction (Regression)

- **Goal:** Predict a student's numerical mood score (\$1.0\$ to \$5.0\$) based on demographic data and basic self-reported mental health statuses.
- **Target Feature Engineering:** Derives a composite `mood_score` from three features:
$$\text{mental_health_score} = (\text{depression} \times 3) + (\text{anxiety} \times 2) + (\text{panic} \times 1)$$
$$\text{mood_score} = 5 - \left(\frac{\text{mental_health_score}}{6} \times 4\right)$$

This maps a student with no depression, anxiety, or panic to a score of 5.0 (Excellent), and a student experiencing all three to 1.0 (Very Low).

Algorithms Evaluated & Performance (Actual Run):

- **Linear Regression (Best Model):** R^2 : 1.000 | MAE: 0.000 (Saved)
- **Gradient Boosting Regressor:** R^2 : 1.000 | MAE: 0.000
- **Random Forest Regressor:** R^2 : 1.000 | MAE: 0.011

- **Technical Note:** Why is the R^2 a perfect 1.000? Because the target `mood_score` is a deterministic linear combination of three input features: `has_depression`, `has_anxiety`, and `has_panic`. Since these three features are directly included in the model's inputs, a Linear Regressor mathematically reproduces the target equation with zero error.
- **Output:** The model with the highest R^2 score is saved as `mood_predictor.pkl`, and its corresponding `StandardScaler` is saved as `mood_scaler.pkl`.

Task 2: Sentiment Classifier (NLP / Classification)

- **Goal:** Classify short text statements from students into one of three sentiment levels: `NEUTRAL`, `NEGATIVE`, or `CRISIS`.
- **Data Combination:** Merges `Combined_Data.csv` and `mental_health.csv`. It maps clinical status categories and binary crisis labels into unified text/sentiment columns.

Preprocessing:

- Cleans text by converting to lowercase and stripping non-alphabetic characters.
- Vectorizes text using **TF-IDF Vectorization** (ngram range 1-2, maximum of 5,000 features, English stopwords exclusion).

Algorithms Evaluated & Accuracy (Actual Run):

- **Logistic Regression (Best Model):** Accuracy: 0.818 (Saved)
- **Support Vector Classifier (SVC):** Accuracy: 0.817
- **Random Forest Classifier:** Accuracy: 0.810

Classification Report (Best Model - Logistic Regression):

- **CRISIS:** Precision: 0.78 | Recall: 0.75 | F1-Score: 0.77
- **NEGATIVE:** Precision: 0.80 | Recall: 0.75 | F1-Score: 0.78
- **NEUTRAL:** Precision: 0.86 | Recall: 0.92 | F1-Score: 0.89
- **Output:** The model with the highest accuracy is saved as `sentiment_classifier.pkl` along with `tfidf_vectorizer.pkl` and `sentiment_label_encoder.pkl`.

Task 3: Sleep Insights (Rule-based & Correlations)

- **Goal:** Group statistical correlations and set up thresholds for personalized advice.
- **Output:** Extracts a Pearson correlation matrix from the numerical features (sleep duration, sleep quality, stress levels) and saves it to `sleep_correlations.json`. It also saves pre-defined warning thresholds and corresponding actionable recommendations to `sleep_insights.json`.

Task 4: FAQ Knowledge Base (Keyword Search Index)

- **Goal:** Convert the static FAQ dataset into a lightweight searchable keyword index.
- **Output:** Strips stopwords and short words from each question, leaving high-value keywords. Saves the indexed dictionary to `faq_knowledge_base.json`.

■ 3. FastAPI Service Integration

The web backend (built with FastAPI in `ml-service/main.py`) exposes these trained models via several endpoints (`ml-service/routers/predict.py`):

`/api/ml/predict-mood (POST)`

- Takes: Student parameters (age, gender, cgpa, marital status, anxiety/depression/panic checkboxes).
- Action: Standardizes features using the saved scaler, predicts the numeric mood score, and returns the score plus a qualitative label (Excellent, Good, Neutral, Low, Very Low).

`/api/ml/analyze-sentiment (POST)`

- Takes: Journal text or a user statement.
- Action: Cleans the text, runs TF-IDF vectorization, classifies the sentiment using the saved classifier, and calculates a confidence score. If the model outputs `CRISIS`, it flags `requires_crisis_support = True` (triggering emergency resources in the frontend).

`/api/ml/sleep-insights (POST)`

- Takes: Sleep hours, stress level, and sleep quality score.
- Action: Validates inputs against rule-based thresholds in `sleep_insights.json` and outputs a custom calculated sleep score and a set of deduplicated habits, articles, and breathing exercises.

`/api/ml/faq-search (GET)`

- Takes: Search query string.
- Action: Extracts search keywords, performs keyword matching against the FAQ database index, and returns the top 3 most relevant questions and answers.

[!NOTE] In addition to the scikit-learn models, the platform features **Aria** (`/api/ml/chat`), a conversational LLM companion powered by the **Groq API** (`llama-3.1-8b-instant`). It uses a strict system prompt to deliver empathetic, safe, non-diagnostic answers to students.

■ 4. How to Run the Code for a Live Demo

If your teacher asks to see the training script run or the API in action, follow these steps in your terminal:

Step 1: Run the Model Training Script

Open a terminal in the project directory, navigate to `ml-service`, and run the training pipeline:

```
cd ml-service
python training/train_all_models.py
```

This will print the R^2 scores, accuracies, classification reports, and verify that the `.pkl` and `.json` files have been written.

Step 2: Start the FastAPI Server

Start the local development server for the machine learning microservice:

```
python -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

Once started, you can access the interactive API documentation at: <http://127.0.0.1:8000/docs>. You can use this Swagger UI to test the endpoints live in front of your teacher.

■ 5. Teacher's Q&A Cheat Sheet (Expected Questions)

Be prepared to answer these technical questions during your presentation:

Q1: Why did you scale your features for the Mood Prediction model?

- **A:** Algorithms like linear regression are sensitive to the scale of input features. Since Age has a range of ~18-40, while features like `has_depression` are binary (0 or 1), a model might assign disproportionate weight to age simply due to its scale. Standard scaling transforms features to have a mean of 0 and standard deviation of 1, ensuring fair learning.

Q2: Why did you choose TF-IDF Vectorization instead of simple Bag of Words (CountVectorizer)?

- **A:** Bag of Words simply counts occurrences of words, meaning common words (like "feel", "really", "just") dominate the representation without carrying much unique sentiment. **TF-IDF (Term Frequency-Inverse Document Frequency)** scales down the weight of words that appear frequently across all documents, highlighting words that carry specific emotional weight (like "empty", "sad", "hopeless").

Q3: Explain how your Mood Score is calculated. Why didn't you just predict the raw answer values?

- **A:** Instead of predicting binary statuses separately, we want to assess overall student mood. We engineered a target continuous variable (`mood_score` from 1.0 to 5.0). We weighted depression highest (3) because of its clinical severity, followed by anxiety (2) and panic (1). We then normalized the sum and scaled it so that 5.0 represents a healthy baseline and 1.0 represents high distress.

Q4: Why evaluate Mood Prediction with both R^2 and MAE?

- **A: MAE (Mean Absolute Error)** gives us a direct, interpretable measure of error (e.g., "our model is off by 0.35 points on average"). **R^2 (Coefficient of Determination)** tells us the proportion of variance in the mood score that is predictable from our features, giving us a standardized measure of fit (where 1.0 is perfect and 0.0 is no better than guessing the mean).

Q5: Why did you map different clinical labels (Anxiety, Depression, Suicidal, Personality Disorder) into three sentiment categories (NEUTRAL, NEGATIVE, CRISIS)?

- **A:** The primary goal of the Sentiment Classifier in a student wellness app is triage and safety. Having too many specific classes increases classification difficulty and isn't actionable for the interface. By grouping labels into **NEUTRAL** (standard reflection), **NEGATIVE** (moderate stress needing self-care exercises), and **CRISIS** (severe distress requiring immediate crisis helpline popups), the app can trigger appropriate frontend actions.

Q6: Why did you use `stratify=y_text` in your train-test split for Sentiment Classification?

- **A:** Since the distribution of sentiment classes (Neutral, Negative, Crisis) is imbalanced, a standard random split might lead to a test set with very few crisis instances. Stratification guarantees that the train and test splits maintain the exact same class proportions as the full combined dataset, leading to reliable evaluation.